

Text Data in Business and Economics
University of Basel, 21–25 September 2026
Benjamin W. Arolt, University of Cambridge

9. Generative AI

Outline

Recap: What is a Large Language Model?

Document Embeddings, Retrieval, and Topics

The Model Landscape (September 2026)

Gen AI for YOUR Research

Recap from Lecture 8: the three-step recipe

- ▶ **Step 1 – Pre-training:** a transformer learns to predict the next token on terabytes of text (GPT) – or to fill in masked tokens (BERT)
 - ▶ result: a *base model* that knows language and facts, but only continues text

Recap from Lecture 8: the three-step recipe

- ▶ **Step 1 – Pre-training:** a transformer learns to predict the next token on terabytes of text (GPT) – or to fill in masked tokens (BERT)
 - ▶ result: a *base model* that knows language and facts, but only continues text
- ▶ **Step 2 – Instruction fine-tuning:** supervised learning on millions of (instruction, good response) pairs → the model answers questions instead of continuing them

Recap from Lecture 8: the three-step recipe

- ▶ **Step 1 – Pre-training:** a transformer learns to predict the next token on terabytes of text (GPT) – or to fill in masked tokens (BERT)
 - ▶ result: a *base model* that knows language and facts, but only continues text
- ▶ **Step 2 – Instruction fine-tuning:** supervised learning on millions of (instruction, good response) pairs → the model answers questions instead of continuing them
- ▶ **Step 3 – Reinforcement learning from human feedback:** raters compare responses; the model is tuned toward preferred answers (helpful, harmless . . . and confident-sounding)

Recap from Lecture 8: the three-step recipe

- ▶ **Step 1 – Pre-training:** a transformer learns to predict the next token on terabytes of text (GPT) – or to fill in masked tokens (BERT)
 - ▶ result: a *base model* that knows language and facts, but only continues text
- ▶ **Step 2 – Instruction fine-tuning:** supervised learning on millions of (instruction, good response) pairs → the model answers questions instead of continuing them
- ▶ **Step 3 – Reinforcement learning from human feedback:** raters compare responses; the model is tuned toward preferred answers (helpful, harmless . . . and confident-sounding)
- ▶ Today: what to *do* with the result – embeddings and retrieval over your own corpus, the September 2026 model landscape, and GenAI in the research workflow

Outline

Recap: What is a Large Language Model?

Document Embeddings, Retrieval, and Topics

The Model Landscape (September 2026)

Gen AI for YOUR Research

Recap: clustering (Lecture 4) meets embeddings

- ▶ Lecture 4: k -means / k -medoids / DBSCAN / hierarchical clustering grouped documents by **distance between their vectors**
 - ▶ back then the vector was a row of the document-term matrix (bag-of-words, TF-IDF)

Recap: clustering (Lecture 4) meets embeddings

- ▶ Lecture 4: k -means / k -medoids / DBSCAN / hierarchical clustering grouped documents by **distance between their vectors**
 - ▶ back then the vector was a row of the document-term matrix (bag-of-words, TF-IDF)
- ▶ Nothing in the clustering algorithms cares *where the vector comes from*
 - ▶ replace the sparse TF-IDF row by a **dense embedding** and the same k -means code runs – but now “distance” means semantic similarity, not word overlap

Recap: clustering (Lecture 4) meets embeddings

- ▶ Lecture 4: k -means / k -medoids / DBSCAN / hierarchical clustering grouped documents by **distance between their vectors**
 - ▶ back then the vector was a row of the document-term matrix (bag-of-words, TF-IDF)
- ▶ Nothing in the clustering algorithms cares *where the vector comes from*
 - ▶ replace the sparse TF-IDF row by a **dense embedding** and the same k -means code runs – but now “distance” means semantic similarity, not word overlap
- ▶ That is the whole recipe for the rest of this section:
document → **embedding** (S-BERT) → cosine similarity / clustering → retrieval (RAG), topics (BERTopic), features for ML

Sentence-BERT (Reimers and Gurevych, 2019)

- ▶ BERT (Lecture 8) gives us a contextual vector for every *token* – but for many research tasks we want one vector per *sentence/document*
 - ▶ document similarity, retrieval, clustering, measuring “distance” between speeches, patents, job ads, court decisions, . . .

Sentence-BERT (Reimers and Gurevych, 2019)

- ▶ BERT (Lecture 8) gives us a contextual vector for every *token* – but for many research tasks we want one vector per *sentence/document*
 - ▶ document similarity, retrieval, clustering, measuring “distance” between speeches, patents, job ads, court decisions, . . .
- ▶ Naive fix: average the token vectors (or take the [CLS] token) → performs poorly on sentence-similarity tasks
- ▶ S-BERT: fine-tune BERT in a **siamese network** so that the pooled embeddings of two sentences are close if they are paraphrases / entail each other (trained on NLI and STS data)

Sentence-BERT (Reimers and Gurevych, 2019)

- ▶ BERT (Lecture 8) gives us a contextual vector for every *token* – but for many research tasks we want one vector per *sentence/document*
 - ▶ document similarity, retrieval, clustering, measuring “distance” between speeches, patents, job ads, court decisions, . . .
- ▶ Naive fix: average the token vectors (or take the [CLS] token) → performs poorly on sentence-similarity tasks
- ▶ S-BERT: fine-tune BERT in a **siamese network** so that the pooled embeddings of two sentences are close if they are paraphrases / entail each other (trained on NLI and STS data)
- ▶ Result: general-purpose **document embeddings** in which cosine similarity is meaningful – the transformer-era successor of averaged word2vec vectors (Lecture 7)

SentenceTransformers and modern embedding models

- ▶ `sentence-transformers` ([sbert.net](https://www.sbert.net/)): Python package for embedding texts and short documents in a few lines of code
 - ▶ hundreds of pre-trained models: paraphrase, semantic similarity, question-answer retrieval; monolingual and multilingual (100+ languages)
 - ▶ runs locally, free, reproducible – good default for research

Sentence Transformers and modern embedding models

- ▶ `sentence-transformers` ([sbert.net](https://www.sbert.net)): Python package for embedding texts and short documents in a few lines of code
 - ▶ hundreds of pre-trained models: paraphrase, semantic similarity, question-answer retrieval; monolingual and multilingual (100+ languages)
 - ▶ runs locally, free, reproducible – good default for research
- ▶ API embedding models (OpenAI `text-embedding-3`, Google Gemini Embedding, Voyage, Cohere): typically stronger, longer inputs (8k+ tokens), but cost money and are not fully reproducible
- ▶ Public leaderboard for choosing a model: MTEB (Massive Text Embedding Benchmark)

Sentence Transformers and modern embedding models

- ▶ `sentence-transformers` ([sbert.net](https://www.sbert.net/)): Python package for embedding texts and short documents in a few lines of code
 - ▶ hundreds of pre-trained models: paraphrase, semantic similarity, question-answer retrieval; monolingual and multilingual (100+ languages)
 - ▶ runs locally, free, reproducible – good default for research
- ▶ API embedding models (OpenAI `text-embedding-3`, Google Gemini Embedding, Voyage, Cohere): typically stronger, longer inputs (8k+ tokens), but cost money and are not fully reproducible
- ▶ Public leaderboard for choosing a model: MTEB (Massive Text Embedding Benchmark)
- ▶ Typical uses in economics:
 - ▶ similarity/distance measures (Lecture 4) with semantics instead of word overlap
 - ▶ features for supervised learning (Lecture 5)
 - ▶ retrieval (RAG) and embedding-based topic models → next slides

Retrieval-Augmented Generation (RAG): dense retrieval

- ▶ Lecture 4 preview: RAG = **retrieve** relevant documents, then let the LLM **generate** an answer conditioned on them (with citations)
- ▶ Modern pipeline:
 1. **Chunk** the corpus (e.g. 200–500-token passages) and **embed** each chunk with a sentence-embedding model (previous slides)
 2. Store vectors in a **vector database** (FAISS, Chroma, pgvector, Pinecone)
 3. **Embed the query** with the same model; nearest-neighbour search (cosine) returns the top- k chunks
 4. Put query + chunks into the LLM prompt → grounded answer

Retrieval-Augmented Generation (RAG): dense retrieval

- ▶ Lecture 4 preview: RAG = **retrieve** relevant documents, then let the LLM **generate** an answer conditioned on them (with citations)
- ▶ Modern pipeline:
 1. **Chunk** the corpus (e.g. 200–500-token passages) and **embed** each chunk with a sentence-embedding model (previous slides)
 2. Store vectors in a **vector database** (FAISS, Chroma, pgvector, Pinecone)
 3. **Embed the query** with the same model; nearest-neighbour search (cosine) returns the top- k chunks
 4. Put query + chunks into the LLM prompt → grounded answer
- ▶ In practice: **hybrid** retrieval (BM25 + dense) and a **re-ranker** (a cross-encoder scoring query–chunk pairs) beat either alone
- ▶ This is what “chat with your PDFs”, Deep Research, and file-search tools do under the hood – and what you can build over *your* corpus (filings, speeches, court decisions, survey open-ends)

Three ways to give an LLM your documents

- ▶ **Put everything in the prompt** (“long context”) – *the open-book exam with the whole library on the desk*
 - + simplest option; nothing to build
 - only works up to a few hundred pages; you pay for the whole corpus every question; quality drops as prompts get very long

Three ways to give an LLM your documents

- ▶ **Put everything in the prompt** (“long context”) – *the open-book exam with the whole library on the desk*
 - + simplest option; nothing to build
 - only works up to a few hundred pages; you pay for the whole corpus every question; quality drops as prompts get very long
- ▶ **Retrieve first, then ask** (RAG) – *a librarian hands you the 5–10 relevant pages*
 - + scales to millions of documents; cheap per question; the answer can **cite** the passages it used
 - a bad search gives a bad answer – the retrieval step (chunking, embedding model, k) needs testing

Three ways to give an LLM your documents

- ▶ **Put everything in the prompt** (“long context”) – *the open-book exam with the whole library on the desk*
 - + simplest option; nothing to build
 - only works up to a few hundred pages; you pay for the whole corpus every question; quality drops as prompts get very long
- ▶ **Retrieve first, then ask** (RAG) – *a librarian hands you the 5–10 relevant pages*
 - + scales to millions of documents; cheap per question; the answer can **cite** the passages it used
 - a bad search gives a bad answer – the retrieval step (chunking, embedding model, k) needs testing
- ▶ **Retrain the model on your documents** (fine-tuning) – *studying before the exam*
 - + learns your task, format, and jargon; a small model can match a big one on a narrow task
 - does *not* reliably memorise facts; no citations; needs labelled data and compute

Three ways to give an LLM your documents

- ▶ **Put everything in the prompt** (“long context”) – *the open-book exam with the whole library on the desk*
 - + simplest option; nothing to build
 - only works up to a few hundred pages; you pay for the whole corpus every question; quality drops as prompts get very long
- ▶ **Retrieve first, then ask** (RAG) – *a librarian hands you the 5–10 relevant pages*
 - + scales to millions of documents; cheap per question; the answer can **cite** the passages it used
 - a bad search gives a bad answer – the retrieval step (chunking, embedding model, k) needs testing
- ▶ **Retrain the model on your documents** (fine-tuning) – *studying before the exam*
 - + learns your task, format, and jargon; a small model can match a big one on a narrow task
 - does *not* reliably memorise facts; no citations; needs labelled data and compute

Rule of thumb: <200 pages $\rightarrow$ prompt; large corpus or need for sources $\rightarrow$ RAG; a specialised classifier $\rightarrow$ fine-tune (or plain supervised ML, Lecture 5). For research, RAG has one big advantage: the evidence is explicit, so it can be **checked and replicated**.

BERTopic: Topic Models from Embeddings (Grootendorst 2022)

- ▶ Idea: replace the bag-of-words input of LDA with **contextual document embeddings** – capturing semantics, word order, and polysemy

BERTopic: Topic Models from Embeddings (Grootendorst 2022)

- ▶ Idea: replace the bag-of-words input of LDA with **contextual document embeddings** – capturing semantics, word order, and polysemy
- ▶ Pipeline:
 1. **Embed** each document with a pretrained transformer (e.g. Sentence-BERT, this lecture)
 2. **Reduce** the dimensionality of the embeddings (UMAP)
 3. **Cluster** the documents (HDBSCAN) → each cluster is a topic (outliers can stay unassigned)
 4. **Label** each topic with **c-TF-IDF**: pool all documents in a cluster into one “class” and apply TF-IDF to extract its most distinctive words

BERTopic: Topic Models from Embeddings (Grootendorst 2022)

- ▶ Idea: replace the bag-of-words input of LDA with **contextual document embeddings** – capturing semantics, word order, and polysemy
- ▶ Pipeline:
 1. **Embed** each document with a pretrained transformer (e.g. Sentence-BERT, this lecture)
 2. **Reduce** the dimensionality of the embeddings (UMAP)
 3. **Cluster** the documents (HDBSCAN) → each cluster is a topic (outliers can stay unassigned)
 4. **Label** each topic with **c-TF-IDF**: pool all documents in a cluster into one “class” and apply TF-IDF to extract its most distinctive words
- ▶ vs. LDA:
 - + number of topics is data-driven; handles short texts well; often more coherent topics
 - usually assigns *one* topic per document (hard clustering, not a mixture); more compute; not a generative model

Outline

Recap: What is a Large Language Model?

Document Embeddings, Retrieval, and Topics

The Model Landscape (September 2026)

Gen AI for YOUR Research

OpenAI today: GPT-5.6 family and GPT-6 Astra (September 2026)

- ▶ **GPT-5.6 (July 2026):** one family, three sizes – **Sol** (flagship), **Terra**, **Luna** (cheap/fast). Same specs across sizes: $\approx$ 1M-token context, 128k max output, text + image input, text output.
- ▶ **GPT-6 Astra (3 Sept 2026):** new flagship for coding, computer use and long agentic tasks; knowledge cutoff 30 Apr 2026; API price \$10 / \$50 per 1M input / output tokens ($2.5\times$ Sol).
 - ▶ Staged rollout: first to vetted organisations (“Trusted Access”/cyber-safety concerns), then Plus/Pro/Business/Enterprise and API “over the coming days”.
- ▶ **Reasoning control:** `reasoning.effort` $\in$ {none, low, medium, high, xhigh, max} – more effort = better answers, more tokens, more cost.
- ▶ **In ChatGPT:** Free/Go default to Luna, Plus/Pro get Sol (and Astra as it rolls out).
- ▶ **Lesson for researchers:** the “model” you use in a chat app changes without notice $\rightarrow$ for research, pin a dated model snapshot via the API and record it.

Coding agents in the stack: Claude Code, Codex, Gemini CLI

Foundation model (Claude / GPT / Gemini) → **Product surface** (app/API) → **Agent tool** (runs actions)

- ▶ **Claude Code (Anthropic):** terminal/IDE agent that *reads your codebase, edits files, runs commands*; included in paid Claude plans (Pro and up); strongest at multi-file implementation, has a “plan mode” separating reasoning from execution.
- ▶ **Codex (OpenAI):** open-source terminal agent (Rust) included in all ChatGPT tiers (even Free); strong for code review (@codex review on GitHub pull requests) and CI integration.
- ▶ **Gemini CLI (Google):** Apache-2.0 open-source agent with 1M-token context; free consumer access ended June 2026 (successor: Antigravity CLI), paid API keys still work.
- ▶ All three speak the **Model Context Protocol (MCP)** – a standard for plugging tools/data (databases, Stata/R, GitHub, Zotero, ...) into agents.
- ▶ **Lecture takeaway:** Claude/GPT/Gemini are *models*; Claude Code/Codex/Gemini CLI are *agent products* that add permissions + tool execution (and therefore add security/reproducibility risks).

Choose your GenAI model (updated 4 September 2026)

Model (provider)	Chat plan, \$/month	API, \$ per 1M tokens in / out	Context / max output	Knowledge cutoff	Images / PDF upload
GPT-6 Astra (OpenAI)	Plus 20 / Pro 100–200 (rolling out)	10 / 50	1M / 128k	04/2026	yes / yes
GPT-5.6 Sol, Terra, Luna (OpenAI)	free, Go 8 (Luna); Plus 20 (Sol)	4/20; 2/12; 0.20/1.20	1M / 128k	02/2026	yes / yes
Claude Fable 5.1 (Anthropic)	Max 100–200	10 / 50	1M / 128k	06/2026	yes / yes
Claude Opus 5, Sonnet 5 (Anthropic)	free (Sonnet); Pro 20; Max 100–200	5/25; 2/10	1M / 128k	05/2026; 01/2026	yes / yes
Gemini 3.8 / 3.7 Flash (Google)	free; AI Plus 8; AI Pro 20; Ultra 100–200	0.75 / 3.75 (1.50 / 7.50 from 2027)	1M / 64k	n. s.	yes (+audio, video) / yes
Gemini 3.1 Pro (Google)	AI Pro and above	2 / 12	1M / 64k	01/2025	yes (+audio, video) / yes
DeepSeek-V4 Pro, Flash (open weights, MIT)	free web/app	0.66/1.98; 0.22/0.66	1M / 384k	n. s.	partly / app only
Qwen 3.6, GLM-5.2, Kimi K2.7, Gemma 4, Mistral (open weights)	self-host, or hosting APIs	mostly <1 / <3	128k–1M	varies	mostly / no

Provider pricing/model pages, checked 4 Sept 2026; standard API rates (batch/cached-input discounts of 50–90% apply on top). n. s. = not stated. Gemini 3.5 Pro (2M context) announced May 2026, still not generally available; Meta's Llama 4 overtaken by Chinese open-weight models.

⇒ Beyond raw quality, choose on **ecosystem**: connectors/MCP, coding agents, PDF/OCR tooling, **data-protection and admin controls** (confidential microdata!).

What does it cost to run an LLM over a corpus? (back-of-the-envelope)

- ▶ Typical text-as-data task: classify / extract from **100,000 documents**, ≈ 500 tokens each, short structured answer (≈ 20 tokens)
 - ▶ $\Rightarrow$ 50M input tokens + 2M output tokens

What does it cost to run an LLM over a corpus? (back-of-the-envelope)

- ▶ Typical text-as-data task: classify / extract from **100,000 documents**, ≈ 500 tokens each, short structured answer (≈ 20 tokens)
 - ▶ $\Rightarrow$ 50M input tokens + 2M output tokens
- ▶ Standard API prices (September 2026), before batch/caching discounts:

Model	\$ / 1M in / out	Cost of the job
DeepSeek-V4 Flash	0.22 / 0.66	$\approx$ \$12
GPT-5.6 Luna	0.20 / 1.20	$\approx$ \$12
Gemini 3.8 Flash	0.75 / 3.75	$\approx$ \$45
Claude Sonnet 5	2 / 10	$\approx$ \$120
GPT-5.6 Sol	4 / 20	$\approx$ \$240
Claude Fable 5.1 / GPT-6 Astra	10 / 50	$\approx$ \$600

What does it cost to run an LLM over a corpus? (back-of-the-envelope)

- ▶ Typical text-as-data task: classify / extract from **100,000 documents**, ≈ 500 tokens each, short structured answer (≈ 20 tokens)
 - ▶ $\Rightarrow$ 50M input tokens + 2M output tokens

- ▶ Standard API prices (September 2026), before batch/caching discounts:

Model	\$ / 1M in / out	Cost of the job
DeepSeek-V4 Flash	0.22 / 0.66	$\approx$ \$12
GPT-5.6 Luna	0.20 / 1.20	$\approx$ \$12
Gemini 3.8 Flash	0.75 / 3.75	$\approx$ \$45
Claude Sonnet 5	2 / 10	$\approx$ \$120
GPT-5.6 Sol	4 / 20	$\approx$ \$240
Claude Fable 5.1 / GPT-6 Astra	10 / 50	$\approx$ \$600

- ▶ A **50 $\times$ price range** for the same job – so: validate a small model against a hand-coded sample (Lecture 5: precision/recall) before paying for the big one; use the big model to *label training data* for a cheap classifier, not to label everything.
- ▶ Reasoning effort multiplies output tokens (often 5–20 $\times$) $\rightarrow$ switch it off for simple classification.

Outline

Recap: What is a Large Language Model?

Document Embeddings, Retrieval, and Topics

The Model Landscape (September 2026)

Gen AI for YOUR Research

Generative AI for Economic Research

- ▶ Korinek (2025)¹:
 - ▶ Provides an overview of how researchers can exploit these technologies

¹The slides for this part follow the outline provided by Korinek (2025)

Generative AI for Economic Research

- ▶ Korinek (2025)¹:
 - ▶ Provides an overview of how researchers can exploit these technologies
 - ▶ Discusses the potential productivity-enhancing effects of cognitive automation

¹The slides for this part follow the outline provided by Korinek (2025)

Generative AI for Economic Research

- ▶ Korinek (2025)¹:
 - ▶ Provides an overview of how researchers can exploit these technologies
 - ▶ Discusses the potential productivity-enhancing effects of cognitive automation
 - ▶ Discusses long-run implications of AI for economic research

¹The slides for this part follow the outline provided by Korinek (2025)

Generative AI for Economic Research

- ▶ Korinek (2025)¹:
 - ▶ Provides an overview of how researchers can exploit these technologies
 - ▶ Discusses the potential productivity-enhancing effects of cognitive automation
 - ▶ Discusses long-run implications of AI for economic research
- ▶ Main takeaway on LLMs for research:

“employ AI agents in research is to treat them like a PI would treat a team of research assistants: they require careful planning of what is to be done, oversight during execution, and detailed vetting of the final results.”

¹The slides for this part follow the outline provided by Korinek (2025)

Generative AI for Economic Research

- ▶ Korinek (2025)¹:
 - ▶ Provides an overview of how researchers can exploit these technologies
 - ▶ Discusses the potential productivity-enhancing effects of cognitive automation
 - ▶ Discusses long-run implications of AI for economic research
- ▶ Main takeaway on LLMs for research:

“employ AI agents in research is to treat them like a PI would treat a team of research assistants: they require careful planning of what is to be done, oversight during execution, and detailed vetting of the final results.”
- ▶ Good practices:
 - ▶ Provide context (e.g., specify point of view, audience, etc.)
 - ▶ Iterate
 - ▶ Be patient

¹The slides for this part follow the outline provided by Korinek (2025)

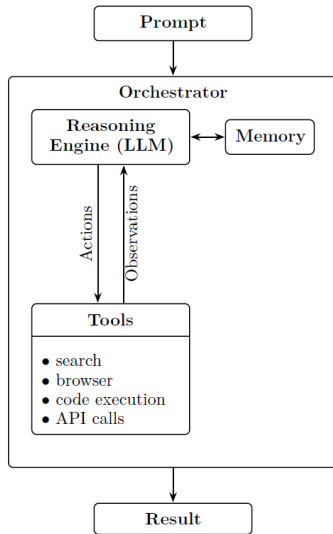
Three paradigms: Chatbots → Reasoners → Agents

- ▶ **Traditional LLMs** (broadly available since Nov 2022): fast, fluent, single-pass responses.
- ▶ **Reasoning models** (introduced Sep 2024, e.g. OpenAI o1): step-by-step analysis for harder math/logic/coding.
- ▶ **Agentic chatbots** (consumer tools from early 2025, e.g. OpenAI Operator): language + reasoning + *autonomous use of external tools*.
- ▶ By 2026: reasoning is a standard dial in all flagship models (GPT-5.6/GPT-6, Gemini 3.x, Claude Opus 5/Fable 5.1) – they auto-select how long to “think”; the frontier has moved to **long-horizon agents** (hours of autonomous coding, computer use, deep research).

What is an AI agent? (Working definition)

- ▶ An AI agent combines:
 - ▶ a large language model (LLM),
 - ▶ planning (decide steps),
 - ▶ memory (track intermediate results),
 - ▶ tool access (web search, code execution, APIs, files).
- ▶ Unlike a standard chatbot, an agent can:
 - ▶ decompose a question into subtasks,
 - ▶ gather information from multiple sources,
 - ▶ execute code and iterate based on intermediate findings,
 - ▶ synthesize results into coherent narratives.

AI Agent Architecture



Comparison of LLM model types

Table 1
Primary Usefulness of LLM Model Types for Research Tasks

Research Category	Traditional LLMs	Reasoning Models	Agentic Chatbots
Ideation & Feedback	Good for initial brainstorming.	Best for structured feedback and identifying logical flaws.	Actively scans literature for novelty and grounding.
Writing	Excellent for drafting, summarizing, and rephrasing existing text.	Ensures logical flow in complex arguments.	Incorporates real-time web information.
Background Research	<i>Shortcoming: No live web access.</i>	Synthesizes info from provided texts.	Best for live web searches and up-to-date literature reviews.
Coding	Generates basic code snippets.	Excellent for writing and debugging complex algorithms.	Executes code, tests hypotheses, and interacts with data files.
Data Analysis	<i>Shortcoming: Cannot execute code.</i>	Helps interpret data and suggest approaches.	Best for end-to-end analysis: data cleaning, coding, visualization.
Math	<i>Shortcoming: Unreliable for complex math.</i>	Solves multi-step problems and formal proofs at PhD level.	Can leverage external computational tools to ensure accuracy.
Promoting Research	Drafts initial promotional content.	Tailors summaries for specific audiences.	Automates creating summaries and posts for multiple platforms.

Note: The most useful model type in each research category is bolded.

Deep Research agents: multi-agent literature review at scale

- ▶ Core architecture:
 - ▶ **Lead/orchestrator** plans the strategy and decomposes the question into subtasks.
 - ▶ **Specialized subagents** investigate threads in parallel (each with its own context/tools).
 - ▶ **Synthesis** integrates findings into a cited report.
- ▶ Strength: compresses time-intensive information gathering + initial synthesis dramatically.
- ▶ Important caveats:
 - ▶ Often compiles existing knowledge (not genuinely novel insight),
 - ▶ Can struggle to identify the most impactful frontier papers,
 - ▶ Steering depth/breadth is constrained by provider cost limits and usage caps.

Coding agents & “vibe coding”

- ▶ Coding agents: terminal-based assistants that can generate, modify, test, and debug multi-file projects.
- ▶ “Vibe coding”: building software end-to-end from natural-language prompts.
- ▶ Example workflow (econometrics):
 - ▶ build a small UI for CSV upload,
 - ▶ select variables for OLS,
 - ▶ compute regression stats (SEs, t -stats, p -values, R^2),
 - ▶ plot scatter + regression line,
 - ▶ iterate quickly via follow-up prompts (fix missing values, fix distributions, etc.).
- ▶ Broader use: even if you hand-code, these tools are strong for code review and bug-finding.

Under the hood: the basic agent loop

- ▶ A simple agent can be structured as:
 1. **Think:** decide what information/tool calls are needed,
 2. **Act:** call tools/APIs (e.g., data retrieval),
 3. **Observe:** parse/validate returned outputs,
 4. **Respond:** produce a grounded natural-language answer.
- ▶ Key design idea: make intermediate outputs explicit so they can be checked, logged, and debugged.
- ▶ Economists: this pattern supports reproducible pipelines (data access → analysis → reporting).

Limitations

Practical limitations (why human oversight remains essential):

- ▶ hallucinations (incl. bogus citations/statistics),
- ▶ error propagation / multi-step cascades,
- ▶ brittleness to prompt changes (reproducibility issues),
- ▶ prompt injection risks when tools/web are involved,
- ▶ not yet reliable as independent economic reasoners.

Example: Pairwise Comparisons – Arold et al. (2025)

- ▶ Recall the union contracts analysis: determining which clauses are pro-worker is not straightforward...

Example: Pairwise Comparisons – Arold et al. (2025)

- ▶ Recall the union contracts analysis: determining which clauses are pro-worker is not straightforward...
- ▶ ...deciding between two clauses which one is more pro-worker is a much easier task!

Example: Pairwise Comparisons – Arold et al. (2025)

- ▶ Recall the union contracts analysis: determining which clauses are pro-worker is not straightforward...
- ▶ ...deciding between two clauses which one is more pro-worker is a much easier task!
- ▶ Have GPT validate pairs of clauses:

“Which of these sentences from a union collective bargaining agreement is more likely to be interpreted as an entitlement, benefit, or amenity for workers? Answer ‘Definitely 1’, ‘Probably 1’, ‘Probably 2’, ‘Definitely 2’, or ‘Neither’. 1. [sentence 1]. 2. [sentence 2].

Example: Pairwise Comparisons – Arold et al. (2025)

- ▶ Recall the union contracts analysis: determining which clauses are pro-worker is not straightforward...
- ▶ ...deciding between two clauses which one is more pro-worker is a much easier task!
- ▶ Have GPT validate pairs of clauses:

“Which of these sentences from a union collective bargaining agreement is more likely to be interpreted as an entitlement, benefit, or amenity for workers? Answer ‘Definitely 1’, ‘Probably 1’, ‘Probably 2’, ‘Definitely 2’, or ‘Neither’. 1. [sentence 1]. 2. [sentence 2].
- ▶ Compute the probability that one clause is more pro-worker than all others
- ▶ Validate previous classification

Pairwise Comparisons – Arold et al. (2025)

Table 4: LLM Validation of Worker Rights as Favoring Workers

Clause Type	Clause Frequency (%)	Pro-Worker Frequency (%)
Worker Right	22.9	79.7
Worker Permission	8.4	67.3
Union Right	2.1	65.7
Firm Obligation	24.7	60.7
Manager Right	0.2	58.4
Worker Obligation	20.9	53.1
Manager Obligation	1.7	44.5
Union Permission	2.0	43.2
Worker Prohibition	3.1	41.6
Manager Prohibition	0.1	38.8
Firm Prohibition	1.5	36.5
Firm Permission	3.4	35.5
Manager Permission	0.4	34.3
Union Obligation	7.0	31.3
Union Prohibition	0.6	28.3
Firm Right	0.9	28.2